Redis — Full Notes (Beginner to Advanced)

Level: Beginner → Intermediate → Advanced

Focus: Python, Django, Backend Development, Interview Preparation

---

Chapter 1: What Is Redis?

Redis stands for **Remote Dictionary Server**. এটি একটি open-source, in-memory data store, যা key-value structure ব্যবহার করে data সংরক্ষণ করে।

Redis সাধারণত ব্যবহৃত হয়:

- Caching
- Session storage
- Rate limiting
- Real-time counters
- Leaderboards
- Message queues and streams
- Celery task broker
- Temporary data storage

Redis কেন Fast?

Redis প্রধানত memory (RAM)-তে data রাখে। তাই সাধারণ disk-based database থেকে অনেক ক্ষেত্রে দ্রুত data access করা যায়।

তবে Redis-এর performance নির্ভর করে data structure, workload, persistence settings, network latency এবং configuration-এর ওপর।

Redis vs Traditional Database

Feature	Redis	PostgreSQL
Primary storage	In-memory	Disk-based
Data model	Key-value and data structures	Relational tables

Typical use	Cache, counters, queues	Persistent application data
SQL	No	Yes
Relationships	Native relational joins নেই	Foreign keys and joins
Persistence	Configurable	Durable database by design
Transactions	Redis commands with transaction mechanisms	ACID database transactions

Important: Redis PostgreSQL-এর replacement নয়। অনেক application-এ PostgreSQL permanent data রাখে, আর Redis frequently accessed বা temporary data দ্রুত সরবরাহ করে।

Chapter 2: Installing Redis

Linux / Ubuntu

```
sudo apt update
sudo apt install redis-server
sudo systemctl enable --now redis-server
```

Check status:

```
sudo systemctl status redis-server
```

Test Redis:

```
redis-cli ping
```

Expected output:

```
PONG
```

Docker

```
docker run -d \
  --name redis \
  -p 127.0.0.1:6379:6379 \
  redis:7
```

Connect:

```
docker exec -it redis redis-cli
```

এই local development command-টি production-এর জন্য সরাসরি ব্যবহার না করে authentication, network isolation, persistence এবং resource limits configure করতে হবে।

Windows

Redis শেখার জন্য Windows-এ WSL2 অথবা Docker ব্যবহার করা যেতে পারে।

Official resources:

- https://redis.io/docs/latest/
- https://redis.io/docs/latest/operate/oss_and_stack/install/install-redis/

Chapter 3: Redis CLI Basics

Redis CLI-তে command চালানো যায়।

```
PING
```

```
SET name "Omar"
GET name
```

Output:

```
"Omar"
```

Delete a key:

```
DEL name
```

Check whether a key exists:

```
EXISTS name
```

Check data type:

```
TYPE name
```

Check remaining expiration time:

```
TTL name
```

List matching keys:

```
SCAN 0 MATCH user:* COUNT 100
```

`SCAN` large keyspaces-এ `KEYS *`-এর চেয়ে বেশি উপযোগী। Production-এ indiscriminate `KEYS *` ব্যবহার করা এড়িয়ে চলা উচিত।

Chapter 4: Redis Key-Value Operations

SET and GET

```
SET username "faruk950147"
GET username
```

SET with expiration

```
SET otp:123456 "verified" EX 60
```

এখানে key-টি 60 seconds পরে expire হবে।

Set only if key does not exist

```
SET lock:payment "processing" NX EX 30
```

এটি temporary lock তৈরির basic pattern। Distributed locking-এর জন্য শুধু এই command-এর ওপর নির্ভর না করে lock ownership এবং safe release-এর বিষয়গুলোও বুঝতে হবে।

Numeric operations

```
SET page_views 0
INCR page_views
INCR page_views
GET page_views
```

Result:

```
"2"
```

Other useful commands:

```
INCRBY page_views 10
DECR page_views
```

```
DECRBY page_views 5
```

Redis numeric commands atomicভাবে execute হয়।

Chapter 5: Redis Data Types

Redis শুধু simple strings সংরক্ষণ করে না। এর বেশ কয়েকটি data structure আছে।

Data type	Typical use
String	Cache, counters, tokens
Hash	Object-like fields
List	Ordered sequence, simple queues
Set	Unique values
Sorted Set	Leaderboards, ranking
Stream	Event streams
Bitmap	Compact boolean tracking

HyperLogLog	Approximate unique counts
Geospatial	Location-based data

5.1 Strings

```
SET product:1 "Laptop"
GET product:1
```

5.2 Hashes

```
HSET user:1 name "Omar" email "omar@example.com" age 25
HGET user:1 name
HGETALL user:1
```

Update a field:

```
HSET user:1 age 26
```

Delete a field:

```
HDEL user:1 age
```

5.3 Lists

```
LPUSH tasks "task1"
LPUSH tasks "task2"
RPUSH tasks "task3"
LRANGE tasks 0 -1
```

Remove an item:

```
LPOP tasks
RPOP tasks
```

Lists support queue-like patterns, কিন্তু production task processing-এর জন্য delivery guarantees, retries এবং acknowledgement দরকার হলে dedicated queue system বা Redis Streams-এর উপযুক্ত design বিবেচনা করতে হবে।

5.4 Sets

```
SADD skills Python Django Redis
SMEMBERS skills
SISMEMBER skills Python
SREM skills Django
```

একই member set-এ duplicate হিসেবে যোগ হয় না।

5.5 Sorted Sets

```
ZADD leaderboard 100 "user1"
ZADD leaderboard 250 "user2"
ZADD leaderboard 180 "user3"
```

Highest scores first:

```
ZREVRANGE leaderboard 0 -1 WITHSCORES
```

একটি leaderboard বা ranking system তৈরিতে এটি উপযোগী।

5.6 Streams

```
XADD orders * order_id 101 status created
XRANGE orders - +
```

Streams event log এবং consumer-group based processing-এ ব্যবহার করা যায়।

Chapter 6: Key Expiration and TTL

TTL means **Time To Live**।

```
SET cache:home "homepage data" EX 300
TTL cache:home
```

এই key-টির expiration 300 seconds।

Milliseconds:

```
SET temporary:value "hello" PX 5000
```

Set expiration separately:

```
EXPIRE temporary:value 60
```

Remove expiration:

```
PERSIST temporary:value
```

TTL return values

- Positive number: remaining lifetime in seconds
- `-1` : key exists but has no expiration
- `-2` : key does not exist

কেন TTL দরকার?

- Cache automatically expire হয়
- OTP বা temporary token-এর lifetime সীমিত থাকে
- Temporary data অনির্দিষ্টকাল জমে থাকার ঝুঁকি কমে
- Cached data periodically refresh করা যায়

Security note: Sensitive data-এর expiration থাকলেই সেটি automatically secure হয়ে যায় না। Access control, encryption requirements এবং logging-ও বিবেচনা করতে হবে।

Chapter 7: Redis Persistence

Redis memory-তে data রাখলেও disk-এ persist করার options আছে।

7.1 RDB

RDB নির্দিষ্ট সময় বা condition অনুযায়ী dataset-এর snapshot তৈরি করে।

Advantages:

- Compact snapshots
- Backup এবং restore সহজ
- Restart recovery-এর জন্য সুবিধাজনক

Limitations:

- Last snapshot-এর পরের data হারাতে পারে
- Snapshot তৈরি করার configuration ও workload performance-কে প্রভাবিত করতে পারে

7.2 AOF

AOF (Append Only File) write operations log করে, যা restart-এর পরে data পুনরুদ্ধারে ব্যবহৃত হতে পারে।

Advantages:

- Configured fsync policy অনুযায়ী কম data loss হতে পারে
- Write history replay করে dataset পুনর্গঠন করা যায়

Limitations:

- File size বড় হতে পারে
- Disk I/O প্রয়োজন হয়
- Rewrite এবং fsync policy performance-কে প্রভাবিত করে

RDB vs AOF

Feature	RDB	AOF
Method	Snapshot	Write-operation log
File size	Often smaller	Often larger
Recovery	Load snapshot	Replay log
Data loss window	Depends on snapshot schedule	Depends on fsync policy
Common use	Backups	More frequent write recovery

Configuration version অনুযায়ী ভিন্ন হতে পারে। নিজের Redis version-এর official documentation দেখে settings যাচাই করবে।

Important: Redis cache হলে persistence বন্ধ রাখা উপযুক্ত হতে পারে। কিন্তু Redis-এ critical data থাকলে backup, persistence, replication এবং disaster recovery plan দরকার।

Chapter 8: Memory Management and Eviction

Redis-এর memory সীমাহীন নয়।

Memory limit configure করার উদাহরণ:

```
CONFIG SET maxmemory 256mb
```

এটি runtime configuration পরিবর্তনের example; production-এ deployment configuration অনুযায়ী persistent setting ব্যবহার করতে হবে।

Eviction Policies

`maxmemory` limit-এ পৌঁছালে Redis policy অনুযায়ী keys evict করতে পারে।

Common policies:

- `noeviction`
- `allkeys-lru`
- `allkeys-lfu`
- `volatile-lru`
- `volatile-lfu`
- `allkeys-random`
- `volatile-random`
- `volatile-ttl`

LRU: Least Recently Used

LFU: Least Frequently Used

Caching-এর জন্য `allkeys-lru` বা `allkeys-lfu` উপযোগী হতে পারে। সঠিক policy নির্ভর করে access pattern এবং workload-এর ওপর।

`noeviction` policy-তে memory limit পূর্ণ হলে কিছু write operations error দিতে পারে।

Chapter 9: Redis in Python

Python-এ Redis ব্যবহার করার জন্য `redis` package install করো।

```
pip install redis
```

Connect to Redis

```
import redis

client = redis.Redis(
    host="localhost",
    port=6379,
    db=0,
    decode_responses=True,
)

print(client.ping())
```

`decode_responses=True` দিলে string responses সাধারণত Python `str` হিসেবে পাওয়া যায়।

SET and GET

```
client.set("username", "Omar")
username = client.get("username")
```

```
print(username)
```

Expiration

```
client.set("otp:123", "123456", ex=60)
```

Counter

```
client.incr("page_views")
print(client.get("page_views"))
```

Hash

```
client.hset(
    "user:1",
    mapping={
        "name": "Omar",
        "role": "developer",
    },
)

print(client.hgetall("user:1"))
```

Delete

```
client.delete("username")
```

Error handling

```
import redis

client = redis.Redis(
    host="localhost",
    port=6379,
    db=0,
    decode_responses=True,
    socket_connect_timeout=2,
    socket_timeout=2,
)

try:
    print(client.ping())
except redis.RedisError as exc:
    print(f"Redis operation failed: {exc}")
```

Production-এ error শুধু print না করে appropriate logging, monitoring এবং fallback strategy ব্যবহার করতে হবে।

Chapter 10: Redis Connection Pool

Connection pool connections পুনরায় ব্যবহার করতে সাহায্য করে।

```
import redis

pool = redis.ConnectionPool(
    host="localhost",
    port=6379,
    db=0,
    decode_responses=True,
    max_connections=20,
)

client = redis.Redis(connection_pool=pool)
```

Benefits:

- Reuse connections
- Manage connection limits
- Reduce repeated connection setup overhead

Pool size application concurrency এবং Redis server-এর capacity অনুযায়ী configure করতে হবে।

Chapter 11: What Is Caching?

Caching হলো frequently accessed data দ্রুত পাওয়ার জন্য temporary storage-এ রাখা।

Example:

Without cache:

```
Request → Django → PostgreSQL → Response
```

With cache:

```
Request → Django → Redis cache
                    ├── Cache hit → Response
                    └── Cache miss → PostgreSQL → Cache → Response
```

Cache Hit

Requested data cache-এ পাওয়া গেলে cache hit হয়।

Cache Miss

Requested data cache-এ না থাকলে source database বা service থেকে data আনতে হয়।

Cache Invalidation

Source data পরিবর্তিত হলে stale cached data কীভাবে remove বা update হবে, সেটাই cache invalidation-এর মূল বিষয়।

Chapter 12: Django + Redis Setup

Install packages:

```
pip install django redis django-redis
```

Redis server local environment-এ চালু থাকতে হবে।

settings.py

```
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://127.0.0.1:6379/1",
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.DefaultClient",
        },
        "KEY_PREFIX": "myproject",
        "TIMEOUT": 300,
    }
}
```

এখানে:

- `BACKEND` : Django-এর Redis cache backend
- `LOCATION` : Redis connection URL এবং logical database number
- `KEY_PREFIX` : application cache keys-এ prefix
- `TIMEOUT` : default cache lifetime in seconds

Production-এ credentials বা Redis URL code-এ hardcode না করে environment variables বা secret manager ব্যবহার করবে।

Chapter 13: Django Cache API

Store a value

```
from django.core.cache import cache

cache.set("site_name", "My Website", timeout=300)
```

Retrieve a value

```
site_name = cache.get("site_name", "Default Site")
```

Delete a key

```
cache.delete("site_name")
```

Check or set a value atomically

```
created = cache.add("task:123", "processing", timeout=60)
```

`add()` key আগে থেকে না থাকলে value set করে।

Get or set

```
def get_site_name():
    return "My Website"

site_name = cache.get_or_set(
    "site_name",
    get_site_name,
    timeout=300,
)
```

Concurrent requests-এর ক্ষেত্রে expensive work একবারই হবে—এমন নিশ্চয়তা এই pattern নিজে থেকে দেয় না। প্রয়োজন হলে locking বা single-flight strategy বিবেচনা করবে।

Chapter 14: Function-Level Caching in Django

Django view cache করার example:

```
from django.views.decorators.cache import cache_page
from django.http import JsonResponse

@cache_page(60 * 5)
def public_stats(request):
    return JsonResponse({
        "users": 100,
        "projects": 25,
    })
```

এটি view response 5 minutes cache করার একটি basic example।

Caution: Response যদি user-specific হয় বা authentication, cookies, permissions অথবা request headers-এর ওপর নির্ভর করে, তাহলে cache key ও cache behavior ঠিকভাবে যাচাই করতে হবে।

ভুলভাবে cache করলে একজন user অন্য user-এর data পেয়ে যেতে পারেন।

Chapter 15: Database Query Caching

Example:

```
from django.core.cache import cache
from .models import Product

def get_active_products():
    key = "products:active:v1"

    products = cache.get(key)

    if products is None:
        products = list(
            Product.objects.filter(is_active=True)
            .values("id", "name", "price")
        )
        cache.set(key, products, timeout=300)

    return products
```

এখানে query result cache করা হচ্ছে।

Important considerations

- Query result পরিবর্তিত হলে cache invalidate করতে হবে
- User-specific query হলে key-তে প্রয়োজনীয় identity/filters থাকতে হবে
- বড় queryset cache করলে memory usage বেড়ে যেতে পারে
- Cached data serialize করার সময় data types বিবেচনা করতে হবে

Chapter 16: Cache Invalidation in Django

ধরা যাক product update হওয়ার পর cache stale হয়ে যাচ্ছে।

```
from django.core.cache import cache

def invalidate_product_cache():
    cache.delete("products:active:v1")
```

Product update service-এ:

```
product.name = "New Name"
product.save()
```

```
  invalidate_product_cache()
```

এটি basic pattern। বাস্তব application-এ cache invalidation transaction commit হওয়ার পরে করা গুরুত্বপূর্ণ হতে পারে, যাতে cache-এ এমন data না যায় যা database transaction-এ commit হয়নি।

উদাহরণ:

```
  from django.db import transaction
  from django.core.cache import cache

  def update_product(product):
      with transaction.atomic():
          product.name = "New Name"
          product.save()

          transaction.on_commit(
              lambda: cache.delete("products:active:v1")
          )
```

এটি transaction সফলভাবে commit হওয়ার পরে cache delete করে।

Chapter 17: Django Sessions with Redis

Session-এ user-specific state রাখা যায়। Redis session backend হিসেবে ব্যবহার করা যেতে পারে।

`django-redis` installed থাকলে:

```
  SESSION_ENGINE = "django.contrib.sessions.backends.cache"
  SESSION_CACHE_ALIAS = "default"
```

এটি cache-backed sessions ব্যবহার করে; cache entry হারালে session-ও হারাতে পারে।

Cache-এর ওপর নির্ভর না করে persistent session storage প্রয়োজন হলে database-backed sessions বা উপযুক্ত `cached_db` strategy বিবেচনা করবে।

Security: Session cookies-এর জন্য `SESSION_COOKIE_SECURE`, `SESSION_COOKIE_HTTPONLY` এবং প্রয়োজন অনুযায়ী `SESSION_COOKIE_SAMESITE` configure করবে। HTTPS ব্যবহার করবে।

Chapter 18: Redis as a Celery Broker

Celery background tasks execute করতে পারে। Redis broker হিসেবে কাজ করতে পারে।

Install:

```
  pip install celery redis
```

Project structure:

```
myproject/
├── manage.py
├── myproject/
│   ├── __init__.py
│   ├── settings.py
│   └── celery.py
└── myapp/
    └── tasks.py
```

myproject/celery.py

```
import os

from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "myproject.settings",
)

app = Celery("myproject")

app.config_from_object(
    "django.conf:settings",
    namespace="CELERY",
)

app.autodiscover_tasks()
```

myproject/init.py

```
from .celery import app as celery_app

__all__ = ("celery_app",)
```

settings.py

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/2"
CELERY_RESULT_BACKEND = "redis://127.0.0.1:6379/3"
```

Production-এ credentials environment variables থেকে load করবে।

myapp/tasks.py

```
from celery import shared_task

@shared_task
def add_numbers(a, b):
    return a + b
```

Call task:

```
from myapp.tasks import add_numbers

add_numbers.delay(10, 20)
```

Start worker:

```
celery -A myproject worker --loglevel=info
```

Celery task execution, retries, acknowledgements এবং result persistence-এর behavior Celery configuration এবং broker settings-এর ওপর নির্ভর করে।

Chapter 19: Redis Rate Limiting

Rate limiting কোনো user বা client নির্দিষ্ট সময়ে কতগুলো request করতে পারবে তা নিয়ন্ত্রণ করে।

Example: একটি API key প্রতি minute-এ 10 requests করতে পারবে।

```
import redis

client = redis.Redis(
    host="localhost",
    port=6379,
    db=0,
    decode_responses=True,
)

def allow_request(user_id: str) -> bool:
    key = f"rate-limit:{user_id}"

    count = client.incr(key)

    if count == 1:
        client.expire(key, 60)

    return count <= 10
```

এই example fixed-window rate limit-এর একটি সরল রূপ। এটি atomic increment করে, কিন্তু increment এবং expiration দুটি আলাদা command হওয়ায় crash বা concurrency-related edge case বিবেচনা করতে হবে।

Production implementation-এ Lua script, Redis Functions বা উপযুক্ত atomic pattern ব্যবহার করে increment ও expiration একসঙ্গে নিশ্চিত করা যেতে পারে।

Django REST Framework throttling

DRF-এ built-in throttling configuration করা যায়:

```
REST_FRAMEWORK = {
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {
        "anon": "20/minute",
        "user": "100/minute",
    },
}
```

DRF-এর default cache configuration অনুযায়ী Redis cache ব্যবহার করলে throttling state Redis-এ রাখা সম্ভব। DRF throttling সব ধরনের high-security abuse prevention-এর বিকল্প নয়; প্রয়োজন অনুযায়ী reverse proxy বা API gateway-এর controls-ও ব্যবহার করতে হবে।

Chapter 20: Redis Atomic Operations and Transactions

Redis individual commands atomicভাবে execute করে। তবে একাধিক command নিয়ে তৈরি workflow স্বয়ংক্রিয়ভাবে পুরোপুরি atomic হয় না।

MULTI / EXEC

```
MULTI
SET balance 100
INCR balance
EXEC
```

Redis transaction queued commands-গুলোকে একসঙ্গে execute করে। তবে SQL database-এর মতো rollback semantics ধরে নেওয়া যাবে না।

WATCH

`WATCH` optimistic concurrency control-এ কাজে লাগে। Watched key পরিবর্তিত হলে `EXEC` abort হতে পারে।

Lua scripting

একাধিক Redis operation একসঙ্গে atomicভাবে execute করার জন্য Lua scripting ব্যবহার করা যায়।

Rate limiting, conditional updates এবং coordinated key operations-এর মতো ক্ষেত্রে এটি উপকারী।

Chapter 21: Redis Pub/Sub

Pub/Sub হলো publisher-subscriber messaging pattern।

Publisher

```
PUBLISH notifications "New message"
```

Subscriber

```
SUBSCRIBE notifications
```

Subscriber connected থাকা অবস্থায় message পেতে পারে।

Important: Redis Pub/Sub সাধারণত message persist করে না। Subscriber disconnected থাকলে সে message মিস করতে পারে। Durable event processing প্রয়োজন হলে Redis Streams বা উপযুক্ত message broker বিবেচনা করবে।

Chapter 22: Redis Streams and Consumer Groups

Streams event data store করতে পারে এবং consumer groups দিয়ে event processing ভাগ করা যায়।

Create event:

```
XADD events * type order_created order_id 101
```

Create consumer group:

```
XGROUP CREATE events order-processors 0 MKSTREAM
```

Read new messages:

```
XREADGROUP GROUP order-processors worker-1 COUNT 10 BLOCK 5000 STREAMS events
>
```

Acknowledge processed message:

```
XACK events order-processors 1700000000000-0
```

উপরের `XACK` -এর ID-টি example মাত্র; বাস্তবে `XREADGROUP` থেকে পাওয়া exact message ID ব্যবহার করতে হবে।

Streams-এর consumer groups pending messages track করে। Production-এ pending message recovery, retries, duplicate processing এবং idempotency handle করতে হবে।

Chapter 23: Redis Security

Production Redis instance secure রাখা জরুরি।

Best practices

1. Redis-কে public internet-এ expose করবে না।
2. Private network, firewall এবং access restrictions ব্যবহার করবে।
3. প্রয়োজন অনুযায়ী ACL users এবং least-privilege permissions configure করবে।
4. Strong authentication এবং TLS ব্যবহার করবে, বিশেষত untrusted network-এর ওপর।
5. Secrets source code বা Git repository-তে রাখবে না।
6. Redis commands এবং client versions নিয়মিত update করবে।
7. Memory limits ও connection limits configure করবে।
8. Backup এবং restore process পরীক্ষা করবে।
9. Monitoring ও audit requirements বিবেচনা করবে।
10. Sensitive data cache করার আগে data exposure risk মূল্যায়ন করবে।

Environment variables

```
import os
import redis

client = redis.Redis.from_url(
    os.environ["REDIS_URL"],
    decode_responses=True,
    socket_connect_timeout=2,
    socket_timeout=2,
)
```

Example environment variable:

```
REDIS_URL=redis://localhost:6379/0
```

Production-এ credentials থাকলে secret manager ব্যবহার করা এবং TLS প্রয়োজন অনুযায়ী `rediss://` URL ও certificate configuration করা উচিত।

Chapter 24: Redis Monitoring and Debugging

Server information

```
INFO
```

Memory:

```
INFO memory
```

Clients:

```
INFO clients
```

Statistics:

```
INFO stats
```

Connected clients:

```
CLIENT LIST
```

Slow log:

```
SLOWLOG GET 10
```

Memory usage of a key:

```
MEMORY USAGE cache:home
```

Useful metrics

- Memory usage
- Connected clients
- Cache hit/miss ratio
- Evicted keys
- Expired keys
- Command latency
- Rejected connections
- Replication status
- Persistence status

Monitoring system-এ alerts configure করবে যাতে memory saturation, connection errors এবং latency বৃদ্ধি পেলে দ্রুত জানা যায়।

Chapter 25: Redis Replication

Replication-এ একটি primary node-এর data এক বা একাধিক replica node-এ copy করা হয়।

Typical benefits:

- Read scaling
- Redundancy
- Failover architectures-এর ভিত্তি

তবে replication asynchronous হতে পারে। তাই primary-তে successful write হলেই replica-তে data ইতিমধ্যে পৌঁছেছে—এমন নিশ্চয়তা নেই।

Replication নিজে backup-এর replacement নয়।

Chapter 26: Redis Sentinel and High Availability

Redis Sentinel monitored Redis deployments-এ primary failure detect করতে এবং failover coordinate করতে পারে।

Main responsibilities:

- Monitoring
- Notifications
- Automatic failover
- Configuration discovery

Sentinel deployment-এ multiple Sentinel instances এবং failure quorum সঠিকভাবে configure করা গুরুত্বপূর্ণ।

Redis Cluster এবং Sentinel একই জিনিস নয়: Cluster sharding এবং distributed deployment দেয়; Sentinel সাধারণত non-clustered primary-replica deployment-এর high availability পরিচালনা করে।

Chapter 27: Redis Cluster

Redis Cluster dataset-কে multiple nodes-এ shard করতে পারে।

Key concepts

- Hash slots
- Sharding
- Replicas
- Cluster topology
- Failover
- Multi-key operation limitations

Redis Cluster 16,384 hash slots ব্যবহার করে। Keys-এর hash slot অনুযায়ী data nodes-এ ভাগ হয়। একাধিক key-তে operation চালাতে হলে keys একই hash slot-এ থাকা লাগতে পারে।

Hash tags ব্যবহার করে keys একই slot-এ রাখা যায়:

```
user:{123}:profile
user:{123}:settings
```

এখানে `{123}` অংশটি hash slot নির্ধারণে ব্যবহৃত হয়।

Cluster ব্যবহার করার আগে capacity, availability requirements এবং operational complexity মূল্যায়ন করবে।

Chapter 28: Cache-Aside Pattern

Cache-aside pattern-এ application নিজেই cache lookup এবং database fallback পরিচালনা করে।

```
from django.core.cache import cache
from myapp.models import Product

def get_product(product_id):
    key = f"product:{product_id}"

    product = cache.get(key)

    if product is None:
        product = (
            Product.objects
            .filter(id=product_id)
            .values("id", "name", "price")
            .first()
        )

        if product is not None:
            cache.set(key, product, timeout=300)

    return product
```

Benefits

- Frequently accessed data দ্রুত পাওয়া যায়
- Cache miss হলে database থেকে data আনা যায়
- Application প্রয়োজন অনুযায়ী cache entries manage করতে পারে

Risks

- Cache invalidation কঠিন হতে পারে
- Concurrent cache misses একই query একাধিকবার চালাতে পারে
- Expired data-এর কারণে database load হঠাৎ বাড়তে পারে

Chapter 29: Cache Stampede and Cache Penetration

Cache Stampede

কোনো popular cache key expire হলে অনেক requests একই সময়ে database query করতে পারে।

Possible solutions:

- TTL jitter
- Request coalescing
- Short-lived locks
- Background refresh
- Stale-while-revalidate strategies

TTL jitter example:

```
import random
from django.core.cache import cache

timeout = 300 + random.randint(0, 60)

cache.set("popular:data", {"value": 123}, timeout=timeout)
```

Cache Penetration

যে data database-এ নেই, তার জন্য repeated requests cache miss তৈরি করতে পারে।

Possible solutions:

- Input validation
- Negative caching for confirmed missing results
- Rate limiting
- Query abuse prevention

Negative caching example:

```
from django.core.cache import cache

MISSING = "NOT_FOUND"

def get_cached_value(key):
    value = cache.get(key)

    if value == MISSING:
        return None

    if value is None:
        # Source lookup would occur here.
        pass

    return value
```

এই example-এ actual source lookup যোগ করতে হবে এবং `MISSING` sentinel-এর TTL স্বল্প রাখতে হবে। `None`-কেই missing key এবং missing data—দুই অর্থে ব্যবহার করলে ambiguity তৈরি হতে পারে।

Chapter 30: Redis Performance Optimization

Performance উন্নত করতে:

1. Network latency কমাও।
2. Unnecessary round trips এড়িয়ে চলো।
3. Batch operations বা pipelining ব্যবহার করো।
4. Appropriate data structures বেছে নাও।
5. Large values এবং oversized collections এড়িয়ে চলো।
6. TTL ও eviction policy সঠিকভাবে configure করো।
7. Memory fragmentation এবং usage monitor করো।
8. Slow commands identify করো।
9. Connection pooling ব্যবহার করো।
10. Realistic workload দিয়ে benchmark করো।

Pipeline in Python

```
import redis

client = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True,
)

with client.pipeline() as pipe:
    pipe.set("user:1:name", "Omar")
    pipe.set("user:2:name", "Faruk")
    pipe.get("user:1:name")

    results = pipe.execute()

print(results)
```

Pipeline অনেক command-এর জন্য network round trips কমাতে পারে। সাধারণ pipeline এবং transactional pipeline-এর semantics এক নয়; `pipeline(transaction=False)` ব্যবহার করে non-transactional batching করা যায়।

Chapter 31: Redis in Django REST Framework

ধরা যাক public API-তে product list রয়েছে।

```
from django.core.cache import cache
from rest_framework.response import Response
from rest_framework.views import APIView
from myapp.models import Product

class ProductListAPIView(APIView):
    def get(self, request):
        key = "api:products:v1"

        data = cache.get(key)

        if data is None:
            data = list(
                Product.objects
                .filter(is_active=True)
                .values("id", "name", "price")
            )

            cache.set(key, data, timeout=300)

        return Response(data)
```

Production considerations

- API data public না private, তা যাচাই করো।
- Query parameters cache key-তে অন্তর্ভুক্ত করো।
- Pagination ও ordering সঠিকভাবে handle করো।
- Permission checks bypass হয় কি না পরীক্ষা করো।
- Data update হলে cache invalidation করো।
- Cached payload size ও serialization cost monitor করো।

Chapter 32: Redis Testing

Redis-dependent code test করার সময় isolated test database/cache ব্যবহার করা উচিত।

Django cache test example:

```
from django.core.cache import cache
from django.test import TestCase
```

```
class CacheTest(TestCase):
    def setUp(self):
        cache.clear()

    def test_cache_set_and_get(self):
        cache.set("test:key", "hello", timeout=30)

        self.assertEqual(
            cache.get("test:key"),
            "hello",
        )
```

এই test configured cache backend ব্যবহার করে। Shared Redis instance-এ `cache.clear()` চালালে অন্য tests বা applications-এর data মুছে যেতে পারে। তাই test environment-এ isolated Redis database, unique key prefixes বা dedicated instance ব্যবহার করবে।

What to test

- Cache hit and miss
- Expiration
- Cache invalidation
- Redis connection failure
- Serialization
- Concurrent access
- Rate-limit boundaries
- Background task retry behavior

Chapter 33: Redis Failure and Fallback Strategy

Redis unavailable হলে application কী করবে, তা workload অনুযায়ী ঠিক করতে হবে।

Cache-only Redis

যদি Redis শুধু cache হয়, তাহলে database থেকে fallback করা যেতে পারে—তবে fallback traffic database overload তৈরি করছে কি না দেখতে হবে।

Redis as a critical component

Redis যদি session store, queue broker বা critical coordination system হয়, তাহলে fallback behavior আরও সতর্কভাবে design করতে হবে।

Example

```
import logging
```

```
import redis
from django.core.cache import cache

logger = logging.getLogger(__name__)

def get_cached_data(key):
    try:
        return cache.get(key)
    except redis.RedisError:
        logger.exception("Redis cache access failed")
        return None
```

এই helper failure-কে cache miss হিসেবে treat করে। এটি সব workload-এর জন্য উপযুক্ত নয়। Sensitive authorization state বা required coordination-এর ক্ষেত্রে Redis failure-কে silently ignore করা নিরাপদ নাও হতে পারে।

Chapter 34: Redis Docker Compose for Django

Example development setup:

```
services:
  redis:
    image: redis:7
    command: redis-server --appendonly yes
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  web:
    build: .
    environment:
      REDIS_URL: redis://redis:6379/1
    depends_on:
      redis:
        condition: service_healthy

volumes:
  redis_data:
```

Important notes

- Compose network-এ Django service Redis-কে `redis` hostname দিয়ে access করে।
- Container-এর ভেতরে `localhost` Django container-কেই নির্দেশ করে, Redis container-কে নয়।
- এই example development-oriented।
- Production-এ secret management, network policies, monitoring, backups, resource limits এবং appropriate Redis deployment strategy configure করতে হবে।
- Image tag-টি নিজের supported version ও patch policy অনুযায়ী pin করবে।

Chapter 35: Redis Use Cases in Real Projects

E-commerce

- Product caching
- Shopping cart state
- Inventory-related temporary counters
- Rate limiting
- Background job coordination

Inventory correctness-এর জন্য Redis counter-কে permanent source of truth হিসেবে ব্যবহার করার আগে consistency requirements বুঝতে হবে।

Social media

- Like counters
- Feed caching
- Trending topics
- Temporary notifications
- Ranking systems

SaaS applications

- API throttling
- Session storage
- Dashboard caching
- Job queues
- Temporary verification codes

Analytics

- Event streams

- Real-time counters
- Approximate unique counts
- Rolling metrics

Chapter 36: Redis vs Memcached

Feature	Redis	Memcached
Data structures	Multiple built-in structures	Primarily simple key-value cache
Persistence	Available	Typically cache-only
Pub/Sub and Streams	Available	Not equivalent built-in features
Sorted sets	Available	Not available
Typical use	Cache and broader data patterns	Simple distributed caching

Redis এবং Memcached-এর মধ্যে পছন্দ application requirements, operational familiarity, data structures এবং performance testing-এর ওপর নির্ভর করে।

Chapter 37: Redis Interview Questions

Beginner

1. What is Redis?

Redis is an in-memory data store that supports key-value operations and multiple data structures.

2. Why is Redis fast?

It serves many operations from memory and provides efficient data structures with low-latency access.

3. What is TTL?

TTL is the remaining lifetime of a key before expiration.

4. What is a cache hit?

Requested data cache-এ পাওয়া গেলে cache hit হয়।

5. Redis কি database replace করতে পারে?

কিছু workload-এ Redis primary data store হতে পারে, কিন্তু relational data এবং complex SQL queries-এর জন্য PostgreSQL-এর মতো database প্রয়োজন হতে পারে।

Intermediate

6. What is cache invalidation?

Source data update হলে cached data delete বা refresh করার process।

7. What is cache-aside?

Application প্রথমে cache check করে, cache miss হলে source database থেকে data নিয়ে cache-এ রাখে।

8. What is Redis Pub/Sub?

Publisher-subscriber messaging pattern, যেখানে subscriber connected থাকা অবস্থায় messages পায়।

9. What is Redis persistence?

Restart-এর পর data recover করার জন্য Redis-এর disk-based persistence mechanisms, যেমন RDB এবং AOF।

10. What is eviction?

Memory pressure-এর সময় configured policy অনুযায়ী keys remove করা।

Advanced

11. Redis Cluster কী?

Redis Cluster multiple nodes-এ dataset shard করে এবং replica/failover capabilities দেয়।

12. What is cache stampede?

একই cache key expire হওয়ার পর অনেক requests একসঙ্গে source database-এ চলে গেলে cache stampede হয়।

13. Why use Lua scripts?

একাধিক Redis operation server-side atomicভাবে execute করতে।

14. Redis Pub/Sub এবং Streams-এর পার্থক্য কী?

Pub/Sub সাধারণত ephemeral messaging; Streams persisted entries এবং consumer-group processing support করে।

15. Redis কি সবসময় durable?

না। Durability persistence settings, replication, storage reliability এবং recovery design-এর ওপর নির্ভর করে।

Chapter 38: Practical Redis Projects

Beginner থেকে advanced পর্যন্ত এই projects তৈরি করো:

Project 1: Basic Cache

- Python client দিয়ে Redis connect করো
- Data set/get করো
- TTL ব্যবহার করো
- Missing keys handle করো

Project 2: Django Product Cache

- Product list API তৈরি করো
- Redis-এ response data cache করো
- Product update হলে cache invalidate করো
- Cache hit/miss log করো

Project 3: API Rate Limiter

- User-based limit implement করো
- Redis atomic operations ব্যবহার করো
- Expiration handle করো
- Concurrent requests test করো

Project 4: Background Tasks

- Celery ও Redis configure করো
- Email বা report-generation task তৈরি করো
- Retry ও failure handling যোগ করো

Project 5: Leaderboard

- Sorted Sets দিয়ে ranking তৈরি করো
- Score update করো
- Top 10 users retrieve করো

Project 6: Event Processing

- Redis Streams-এ events লিখো
- Consumer group তৈরি করো
- Messages process এবং acknowledge করো
- Retry ও duplicate processing handle করো

Chapter 39: Recommended Learning Roadmap

Week 1 — Fundamentals

- Redis installation
- CLI commands
- Strings, hashes, lists, sets
- Sorted sets
- Expiration and TTL

Week 2 — Python

- `redis` package
- Connection pool
- Error handling
- Pipelines
- Atomic commands

Week 3 — Django

- Django cache framework
- `django-redis`
- Query caching
- View caching
- Cache invalidation

Week 4 — Backend Patterns

- Session storage
- Rate limiting
- Celery broker
- Cache stampede

- Testing

Week 5 — Advanced Redis

- Persistence
- Memory policies
- Pub/Sub
- Streams
- Replication and Sentinel
- Cluster fundamentals

Week 6 — Production

- Docker Compose
- Security
- Monitoring
- Failure handling
- Backup and recovery
- Practical project

Chapter 40: Production Checklist

Configuration

- Redis version এবং deployment model নির্ধারণ করা হয়েছে
- Memory limit এবং eviction policy সেট করা হয়েছে
- Connection pool ও timeouts configure করা হয়েছে
- TTL policies নির্ধারণ করা হয়েছে
- প্রয়োজন অনুযায়ী persistence configure করা হয়েছে

Security

- Public access বন্ধ
- Network access সীমিত
- Authentication/ACL configure করা
- Secrets environment variables বা secret manager-এ রাখা
- প্রয়োজন অনুযায়ী TLS configure করা

Reliability

- Monitoring ও alerts চালু
- Backup এবং restore পরীক্ষা করা
- Redis failure behavior নির্ধারণ করা

- Queue consumers-এর retry ও recovery policy আছে
- Cache invalidation test করা

Django

- Redis URL environment থেকে load করা
- Cache keys যথাযথভাবে namespaced করা
- User-specific data নিরাপদভাবে cache করা
- Cache miss এবং backend error handle করা
- Automated tests চালানো

Final Summary

Redis একটি versatile in-memory data store, যা Django backend application-এ caching, sessions, counters, rate limiting এবং background task infrastructure-এর জন্য কাজে লাগে।

একজন Python/Django Backend Developer হিসেবে এই learning order অনুসরণ করো:

1. Redis CLI ও data structures
2. Python `redis` package
3. Django cache framework
4. Cache invalidation ও expiration
5. Rate limiting
6. Celery + Redis
7. Persistence ও memory management
8. Security, monitoring ও failure handling
9. Streams, replication এবং Cluster
10. Production-ready projects

Best practice: PostgreSQL-এ permanent relational data রাখো, Redis-কে উপযুক্ত cache বা coordination layer হিসেবে ব্যবহার করো, এবং production-এর আগে durability, consistency, security ও failure scenarios পরীক্ষা করো.